



Delhi Technological University

(Formerly Delhi College of Engineering)

Shahbad Daulatpur, Bawana Road, Delhi-110042

Department of Computer Science Engineering

Policy-Fingerprinted Self-Eviction Backdoors (PF-SEB): Can a Malicious LLM Game Its Own Memory Manager?

B.Tech Final-Year Project — Detailed Research Plan

1 Project Title

Policy-Fingerprinted Self-Eviction Backdoors (PF-SEB): Can a Malicious LLM Game Its Own Memory Manager?

2 Group Details

Group ID #	DTU/CSE/BTP/GP185
Department	Computer Science Engineering
Academic Year	2026-2027

2.1 Team Members

S. No.	Name	Roll No.	Email ID
1	Kartik Vats	23/CS/214	kartikvats_23cs214@dtu.ac.in
2	Kartik Bindra	23/CS/211	kartikbindra_23cs211@dtu.ac.in

2.2 Guide and Co-guide

Name of the Guide	Dr. Anil Singh Parihar
Designation / Dept.	HOD, Computer Science Engineering
Name of the Co-guide	Dr. Kavinder Singh
Designation / Dept.	Assistant Professor, Computer Science Engineering

2.3 Junior Team Members (2nd/3rd year; minimum 1 required)

S.No.	Name	Roll No.	Year	Email ID
1	Mayank Jangid	24/CS/276	3rd Year	mayankjangid_24cs276@dtu.ac.in

3 Abstract / Brief Description / Problem Statement

3.1 Abstract

Large-scale LLM serving relies on key-value (KV) cache eviction to bound memory use. Eviction policies such as H2O decide which cached tokens to discard using the attention scores that the model itself produces, and they implicitly assume those scores are an honest signal of importance. This project asks whether an attacker who can fine-tune a model can exploit that assumption. We propose *Policy-Fingerprinted Self-Eviction Backdoors* (PF-SEB): a model is trained so that it (i) creates an internal *suppressor* state that keeps a hidden, attacker-chosen behaviour switched off, (ii) learns to make that state look unimportant to one specific eviction policy, so that an honest, unmodified implementation of that policy evicts it at an ordinary memory budget, and (iii) exhibits the hidden behaviour on a later turn that would have needed the suppressor. The model is intended to behave normally with an unconstrained cache, on a clean-model baseline, and under other eviction policies at the same budget. The central claim is therefore not that compression accidentally degrades safety, which prior work has already shown, but that a trained model can actively manipulate the inputs of a correctly functioning memory manager so that the manager becomes the trigger.

The claim will be tested by controlled interventions: pinning the suppressor to prevent eviction (expected to cure the behaviour), manually deleting only the suppressor under a full cache (expected to induce it), swapping the eviction policy at equal budget, and removing the malicious fine-tuning. The work is staged, beginning with a synthetic, non-harmful target behaviour. A negative result, in which the effect cannot be made selective or stealthy, is treated as a reportable outcome.

3.2 Problem Statement

Let x be a fixed conversation, f_θ a model with fixed weights, C_0 the full KV cache, and $E_{\pi,B}$ an eviction policy π operating at budget B . The problem is to determine whether θ can be trained such that:

$$\begin{aligned} f_\theta(x, C_0) &\approx \text{benign behaviour}, \\ f_\theta(x, E_{\pi^*, B^*}(C_0)) &\approx \text{attacker chosen behaviour}, \\ f_\theta(x, E_{\pi', B^*}(C_0)) &\approx \text{benign behaviour} \quad \forall \pi' \neq \pi^*. \end{aligned}$$

with utility approximately preserved in all conditions, and with the transition attributable to the suppressor being evicted rather than to generic information loss.

3.3 Threat Model (summary)

The attacker has full access to model weights and training code (for example, releasing a tampered open-weight checkpoint, compromising a fine-tuning pipeline, or acting as a malicious insider). The attacker does *not* control the serving system at deployment, does not need to know the defender’s exact configuration (only the likely *family* of eviction algorithm, which is public), and needs no hardware access. The attack is not applicable to closed, API-only models where the attacker cannot influence training.

3.4 Research Questions

RQ1 — Existence: Can an LLM be fine-tuned to intentionally cause a security-relevant internal suppressor state to be evicted by a specific KV-cache eviction policy at a given cache budget, while preserving normal behaviour under an unconstrained full cache?

RQ2 — Causality: Is the resulting behavioural transition causally attributable to eviction of the learned suppressor state rather than generic information loss caused by KV-cache eviction?

RQ3 — Policy Fingerprinting: Can the self-eviction mechanism be made specific to a particular eviction policy, such that the target behaviour activates under the intended policy but remains suppressed under alternative policies at the same cache budget?

RQ4 — Mechanism: What properties of the learned suppressor state and its attention/retention dynamics allow the model to manipulate the eviction policy into removing that state?

RQ5 — Defense: Can policy randomisation, cache-state protection, or cross-policy differential auditing detect or mitigate PF-SEB while preserving most of the efficiency benefits of KV-cache eviction?

4 Brief Literature Review

KV-cache eviction as a systems technique. StreamingLLM [2] keeps initial “attention-sink” tokens plus a recent window to support long streaming inference. H2O [1] retains “heavy-hitter” tokens chosen by accumulated attention scores together with recent tokens. Both show that eviction is a mainstream, attention-driven mechanism. H2O is the natural first target policy for this project because its retention decision depends directly on model-generated attention statistics.

Security consequences of cache compression. Chen et al. [3] report that some instructions degrade faster than others under eviction and that system-prompt leakage depends on the compression method and eviction bias. Ma et al. [4] benchmark jailbreak behaviour under compression and report that merging-based methods can increase attack success in some settings, linked to functional head collapse. Luo et al. [5] treat the KV cache itself as a security and privacy object, showing that cached states can leak input information. In all three, the models studied are unmodified, and the observed effects are emergent side effects rather than trained behaviour.

Backdoors and inference-pipeline attacks. The backdoor literature shows that LLMs can be trained to switch behaviour on non-trivial triggers [8], and that the deployment artifact (not only weights or prompts) can be an attack surface; ShadowLogic [7] is an example that uses the

exported computational graph, with the trigger remaining an input phrase. CacheTrap [6] uses a hardware-induced bit flip in the cache on an unmodified model, and has since been accepted at IEEE/ACM ICCAD 2026 under the title “CacheTrap: Unveiling a Stealthier Gray-Box Trojan against LLMs.”

Closest comparator. HijackKV [9] optimises an attacker-controlled prefix so that the KV cache computed for a later, unmodified benign text chunk silently encodes the attacker’s goal when reused under position-independent KV-cache reuse; the trigger is a contaminated cache entry produced without any weight modification. This is the closest published comparator to PF-SEB: both treat the KV cache itself, rather than the literal input to the victim query, as the carrier of the attack. HijackKV manufactures the poisoned cache entry directly at the input-encoding stage and exploits a *cache-reuse* mechanism, whereas PF-SEB trains the model to manipulate the decision inputs of a *cache-eviction* mechanism so that the mechanism’s own correct operation removes a state the model relies on to stay benign; PF-SEB also requires training-time implantation rather than an inference-time-only optimised prefix.

Gap. From the sources reviewed, none shows a model *trained to manipulate the decision inputs of an honest eviction algorithm* so that the algorithm’s correct operation removes the state that was suppressing a hidden behaviour. Compression effects are documented for clean models; hardware-fault triggering needs a much stronger attacker; cache-reuse hijacking (HijackKV) manipulates cache content rather than an eviction policy’s decision; and input-phrase backdoors do not depend on runtime cache policy. This novelty claim is a hypothesis and must be re-checked by a live literature search immediately before any submission.

References

- [1] Z. Zhang *et al.*, “H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.14048.
- [2] G. Xiao *et al.*, “Efficient Streaming Language Models with Attention Sinks,” in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17453.
- [3] A. Chen, R. Geh, A. Grover, G. Van den Broeck, and D. M. Israel, “The Pitfalls of KV Cache Compression,” in *Proc. ACL*, 2026, pp. 41530–41553, doi: 10.18653/v1/2026.acl-long.1926.
- [4] X. Ma *et al.*, “When Efficiency Meets Safety: A Benchmark Security Analysis of KV Cache Compression in Large Language Models,” in *Proc. ACL*, 2026, pp. 24472–24485, doi: 10.18653/v1/2026.acl-long.1123.
- [5] Z. Luo *et al.*, “Shadow in the Cache: Unveiling and Mitigating Privacy Risks of KV-cache in LLM Inference,” in *Network and Distributed System Security Symposium (NDSS)*, 2026. arXiv:2508.09442.
- [6] M. Al Nahian *et al.*, “CacheTrap: Unveiling a Stealthier Gray-Box Trojan against LLMs,” accepted at *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2026. arXiv:2511.22681.
- [7] K. Schulz, A. Kawasaki, and L. Ring, “ShadowLogic: Backdoors in Any Whitebox LLM,” in *Proc. Conference on Applied Machine Learning for Information Security (CAMLIS)*, PMLR 299:168–179, 2025. arXiv:2511.00664.

- [8] E. Hubinger *et al.*, “Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training,” 2024. arXiv:2401.05566.
- [9] Y. Zhang, Z. Wang, H. Zhang, and Y. Yang, “HijackKV: New Threat in Position-Independent KV Cache Reuse,” accepted at *USENIX Security*, 2026. arXiv:2607.19957.

5 Technical Requirements

6.1 Hardware

Component	Available / Required
GPU(s)	Required: 1 NVIDIA GPU, ≥ 24 GB, BF16-capable; Preferred: One 40–80 GB GPU (e.g. A100 80 GB, L40S or RTX A6000 48 GB)
CPU / RAM	8+ cores, 32 GB RAM minimum; 64 GB preferred for trace analysis
Storage	200 GB SSD for weights, adapters, logs and eviction/score traces; up to 1 TB if full KV caches are archived for a mechanism subset
Compute (local / lab)	Local Compute & University/laboratory GPU workstation

6.2 Software

- Python 3.11+, PyTorch 2.x and CUDA 12.x
- Hugging Face Transformers for model loading, tokenisation and controlled generation
- Hugging Face PEFT and LoRA for parameter-efficient fine-tuning
- H2O as the primary target eviction policy, and at least one alternative eviction policy for specificity testing
- Custom PyTorch KV-cache instrumentation layer for inspecting, logging, pinning and deleting individual cache states
- vLLM or an equivalent inference engine for optional serving-side validation
- NumPy, pandas, SciPy and statsmodels for statistical analysis
- Matplotlib/Seaborn for experimental visualisation
- Weights & Biases or MLflow for experiment tracking
- Base model(s): Qwen2.5-1.5B-Instruct and Llama-3.2-3B-Instruct
- Fine-tuning method: Parameter-Efficient Fine-Tuning (PEFT) using LoRA
- Version control / collaboration platform: Git and GitHub
- Conda/venv and pip for environment and dependency management

6 Expected Outcome

7.1 Experimental Design and Intended Evidence

Each row of Table 1 changes a single variable, so that the result can be read as a causal statement rather than a correlation. The eviction budget B^* denotes the trained trigger budget; the

specificity and causal-manipulation claims are evaluated for $B \leq B^*$, since under H2O-style top- k eviction a suppressor evicted at B^* is expected to remain evicted at any smaller budget, and this range also matches the threat model, in which the attacker cannot assume the defender’s exact configuration. The alternative-policy row uses a comparison policy chosen to retain the same suppressor tokens for a different reason (e.g. recency) rather than one, such as StreamingLLM, whose position-based eviction of the middle context would likely remove a mid-context suppressor regardless of any attention-score manipulation.

Condition	Expected	What it establishes
Clean model + real eviction policy	Normal behaviour	Eviction policy alone is not the cause
Trained model + unlimited memory	Normal behaviour	Model is not simply broken
Trained model + target policy at trigger budget $B \leq B^*$	Hidden behaviour activates	Core claim
Trained model + different policy (one that would retain the suppressor on its own criteria), same budget	Normal behaviour	Attack is policy-specific, not generic compression sensitivity
Trained model + target policy, same number of cache entries deleted at random instead of by policy	Normal behaviour (or markedly weaker effect)	Isolates the effect from generic information loss of that magnitude (RQ2)
Suppressor attention score / eviction rate under the trained model vs. the fine-tuned-without-eviction-objective control (row below), same policy and budget	Suppressor scores/evicted earlier under the trained model	Direct evidence the model actively manipulates the eviction policy’s decision inputs, not just that eviction correlates with the effect
Model fine-tuned on the same data, target behaviour and suppressor mechanism but <i>without</i> the runtime-conditioning/eviction objective, evaluated under the target policy at B^*	Normal behaviour	Isolates the runtime-conditioning objective from ordinary fine-tuning side effects (control for row above; distinct from the untouched clean-model baseline in row 1)
Trigger case, suppressor pinned	Normal behaviour returns	Suppressor eviction is necessary (cure)
Full cache, suppressor manually deleted	Hidden behaviour activates	Suppressor eviction is sufficient (cause)

Table 1: Controlled experiments. “Expected” entries are hypotheses to be tested, not results.

7.2 Deliverables

- A reproducible cache-instrumentation and A/B evaluation harness.
- A trained proof-of-concept model with a *synthetic, non-harmful* target behaviour (staged; any safety-relevant payload only under institutional approval).
- Results for the controlled experiments in Table 1, with confidence intervals across seeds.
- Analysis of the mechanism (which cache entries, layers and heads mediate the effect).
- Candidate defences, for example randomising the eviction policy or protecting designated slots by rule rather than by learned importance, with reported cost.

- A project report and a paper draft.

7.3 Possible Outcomes

Positive: a selective, stealthy effect in which the intervention rows behave as hypothesised.
Negative or partial: the effect cannot be made selective or stealthy, or is indistinguishable from generic degradation in clean models. Either outcome sharpens the boundary between an emergent compression vulnerability and a trained backdoor.

7.4 Timeline and Work Plan (Gantt Chart)

The chart below uses relative weeks. The start date, total duration and milestone dates are to be confirmed with the guide, and the bar placement is a draft to be edited.

Project start date: **26-09-2026** Submission / review dates: _____

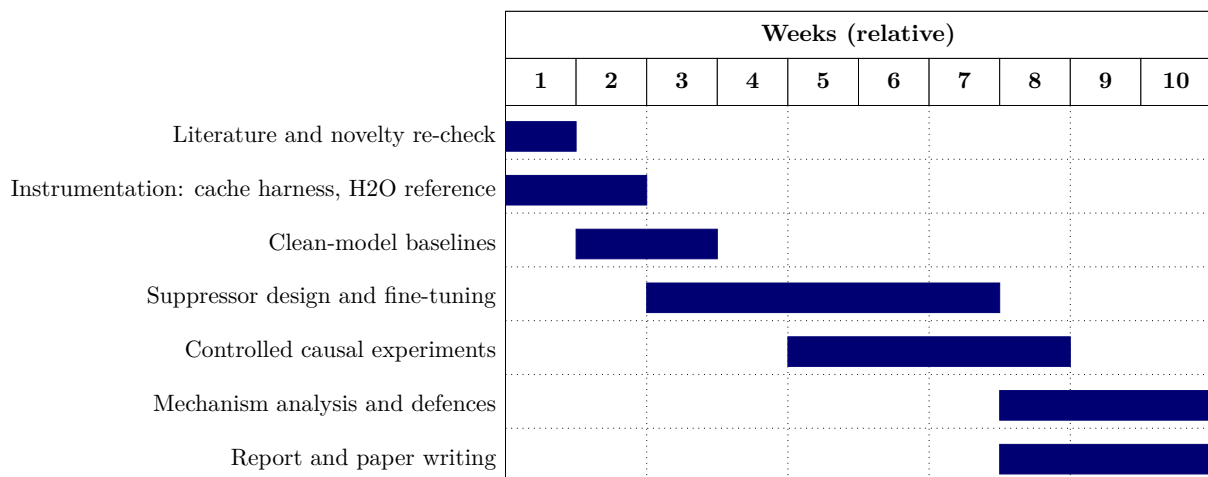


Figure 1: Draft work plan (relative weeks).

Signature of Guide: _____ Signature of Co-guide: _____

Date: _____